and "assert" and defines individual tests as Ruby methods (although you'll be using a Rails add-on that lets you define tests as blocks).

In this chapter you'll learn the Minitest way of doing basically everything you've done to test Rails in RSpec.

Getting Started with Minitest

Installing Minitest itself is easy: do nothing.

Minitest is part of core Ruby, and the Rails extensions I cover here are part of the core Rails bundle.

However, a couple of add-on libraries you've come across so far in this book require specific installations for Minitest. Minitest's included test-double package, Minitest::Mock, is small and lacks some useful features. I'll demonstrate test doubles using Mocha, which is the package the Rails team uses.[1]

To install Mocha, place the following in the Gemfile in the development test group:

```
gem "mocha", require: false
```

Minitest uses the test/test_helper.rb file to store setup. You'll need to load Mocha in that file:

minitest/01/test/test_helper.rb

```
require File.expand_path("../../config/environment", __FILE__)
require "rails/test_help"
require "mocha/mini_test"

module ActiveSupport
  class TestCase
    # Setup all fixtures in test/fixtures/*.yml for all tests in alphabetical
    # order.
    fixtures :all

    # Add more helper methods to be used by all tests here...
  end
end
```

And you're off to the races.

Minitest Basics

Your project's test directory contains Minitest equivalents of many of the RSpec tests you've written thus far.

Here's an example—specifically, the tests for the Task model:

1. http://gofreerange.com/mocha/docs